

RECORD

Field Details

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	4
Student Name	Nagadoleswari Madabathula
Roll No.	A24126552267
Date	30-08-2026

1. TITLE

Dynamic Student List Manager Using JavaScript and DOM Manipulation

2. AIM

To develop a dynamic webpage using JavaScript that accepts a student's Name and Department through a form and allows student records to be added to and removed from a list on the page using DOM manipulation.

3. OBJECTIVE

The objectives of this experiment are:

- To select and manipulate DOM elements using JavaScript.
- To validate user input and alert the user when required fields are left blank.
- To dynamically create a student card element using `createElement()` and set its content.
- To add the dynamically created student card to the webpage using `appendChild()`.
- To attach a Delete button to each student card that removes it from the list when clicked.
- To dynamically show or hide a "No students added" message based on how many students are in the list.

4. THEORY

A dynamic webpage changes its content in response to user interaction, without requiring a page reload. The Document Object Model (DOM) represents an HTML page as a tree of objects that JavaScript can read and modify, and `document.getElementById()` is used to select a specific element from this tree.

`document.createElement()` is used to build new elements entirely from JavaScript, such as a student card `<div>`, its heading and paragraph, and a Delete `<button>`. Once created, these elements are unattached to the page until they are inserted using `parentElement.appendChild(childElement)`.

Event handling connects user actions to code. `addEventListener("click", fn)` registers a function that runs whenever a button is clicked, allowing the Add Student button and each dynamically created Delete button to trigger their own behaviour.

Input validation is performed using `.value.trim()`, which reads an input field's text and strips extra whitespace, so that an empty or whitespace-only entry can be detected and rejected with an `alert()` before a card is created.

Each student card removes itself from the page using `element.remove()` when its own Delete button is clicked. After a removal, `studentList.children.length` is checked so that the "No students added" message can be shown again once the list becomes empty, and hidden whenever it is not.

5. ALGORITHM / PROCEDURE

- Create an HTML file with input fields for Student Name and Department, an Add Student button, a "No students added" message, and an empty container for the student list.
- Add internal CSS to style the form container, the student cards and the Delete buttons.
- Select the name input, department input, Add Student button, student list container and message element using `getElementById()`.
- Attach a click event listener to the Add Student button using `addEventListener()`.
- Inside the handler, read and trim the values of the Name and Department fields using `.value.trim()`.
- If either field is empty, show an alert and stop, without adding a student.
- Otherwise, dynamically create a student card `<div>`, along with a name `<h3>`, a department `<p>` and a Delete `<button>`, using `createElement()`.
- Attach a click event listener to the Delete button that removes its own student card from the page using `remove()`.
- Append the name, department and Delete button to the student card, and the student card to the student list, using `appendChild()`.
- Clear the input fields and hide the "No students added" message.
- When a student card is deleted, check `studentList.children.length` and show the message again if the list is empty.
- Open the HTML file in a browser, add students with different names and departments, and verify that cards can be added and deleted correctly.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Dynamic Student List</title>

  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f2f2f2;
      text-align: center;
      padding: 30px;
    }
  </style>
</head>
<body>
```

```

    }

    .container {
        width: 500px;
        margin: auto;
        background-color: white;
        padding: 25px;
        border-radius: 10px;
    }

    input {
        width: 80%;
        padding: 10px;
        margin: 10px;
    }

    button {
        padding: 10px 20px;
        margin: 10px;
        background-color: blue;
        color: white;
        border: none;
        cursor: pointer;
    }

    .student-card {
        background-color: #e6f0ff;
        padding: 15px;
        margin-top: 15px;
        border-radius: 8px;
    }

    .delete-btn {
        background-color: red;
    }
</style>
</head>

<body>

    <div class="container">

        <h1>Dynamic Student List</h1>

        <input
            type="text"
            id="name"
            placeholder="Enter Student Name"
        >

        <input
            type="text"
            id="department"
            placeholder="Enter Department"
        >

        <br>

        <button id="addButton">Add Student</button>

        <p id="message">No students added.</p>

```

```
<div id="studentList"></div>

</div>

<script>

  const nameInput = document.getElementById("name");
  const departmentInput = document.getElementById("department");
  const addButton = document.getElementById("addButton");
  const studentList = document.getElementById("studentList");
  const message = document.getElementById("message");

  addButton.addEventListener("click", function () {

    const name = nameInput.value.trim();
    const department = departmentInput.value.trim();

    if (name === "" || department === "") {
      alert("Please enter all details!");
      return;
    }

    const studentCard = document.createElement("div");
    studentCard.className = "student-card";

    const studentName = document.createElement("h3");
    studentName.textContent = "Name: " + name;

    const studentDepartment = document.createElement("p");
    studentDepartment.textContent =
      "Department: " + department;

    const deleteButton =
      document.createElement("button");
    deleteButton.textContent = "Delete";
    deleteButton.className = "delete-btn";

    deleteButton.addEventListener("click", function () {
```

```

        studentCard.remove();

        if (studentList.children.length === 0) {
            message.style.display = "block";
        }
    });

    studentCard.appendChild(studentName);
    studentCard.appendChild(studentDepartment);
    studentCard.appendChild(deleteButton);

    studentList.appendChild(studentCard);

    nameInput.value = "";
    departmentInput.value = "";

    message.style.display = "none";
});
</script>
</body>
</html>

```

7. CODE EXPLANATION

Code	Explanation
<code>document.getElementById()</code>	Selects the name input, department input, Add Student button, student list container and the "No students added" message element.
<code>addEventListener("click", fn)</code>	Registers a function that runs whenever the Add Student button is clicked (event handling).
<code>.value.trim()</code>	Reads the text entered in the Name and Department fields and removes leading/trailing whitespace.
<code>if (name === "" department === "")</code>	Validates that both fields are filled in; shows an alert and stops execution if either is blank.
<code>document.createElement()</code>	Dynamically creates the student card <code><div></code> , its <code><h3></code> name, <code><p></code> department and the Delete <code><button></code> .
<code>.textContent</code>	Sets the visible text of the created name, department and delete-button elements.
<code>.className</code>	Assigns the "student-card" and "delete-btn" CSS classes to the newly created elements for styling.
<code>appendChild()</code>	Inserts the name, department and delete button into the student card, and the card into the student list.
<code>studentCard.remove()</code>	Removes a student's card from the DOM when its Delete button is clicked.
<code>studentList.children.length</code>	Checks how many student cards remain after a deletion, to decide whether to show the "No students added" message.
<code>message.style.display</code>	Hides the "No students added" message when a student is added, and

Code	Explanation
	shows it again when the list becomes empty.

8. TEST CASES AND EXECUTION DATA

Each test case was executed by interacting with the application in the browser.

Test Case	Action	Expected Result	Status
TC-01	Page load	"No students added." message is shown; student list is empty	Pass
TC-02	Enter a valid Name and Department, click Add Student	A new student card appears showing Name, Department and a Delete button; input fields are cleared; message is hidden	Pass
TC-03	Click Add Student with Name or Department left empty	An alert "Please enter all details!" is shown; no card is added	Pass
TC-04	Add a second student with different details	New student card is appended below the first, without removing it	Pass
TC-05	Click the Delete button on a student card	That student's card is removed from the list	Pass
TC-06	Delete the last remaining student card	"No students added." message is shown again	Pass

9. OUTPUT

Output 1: Dynamic Student List page on load, with the "No students added." message shown

Dynamic Student List

Add Student

Name: nagadoleswari madabathula

Department: csm

Delete

Name: nani sai

Department: eee

Delete

Output 2: Student list after adding two students (Nagadoleswari Madabathula - CSM, Nani Sai - EEE), each with a Delete button

10. RESULT

Thus, a dynamic webpage was successfully developed using JavaScript, featuring a student registration form that validates its inputs, dynamically creates a student card for each entry using DOM methods, and allows any student card to be removed from the list in real time. The application was verified using multiple test cases.